Assignment

Problem Statement

You are given a small company dataset. Use `MongoDB` locally through **`mongosh`** to complete practical tasks based on the MongoDB Advanced training.

The assignment focuses on the following areas:

- Schema validation
- Advanced array queries and updates
- Bulk write operations
- Advanced aggregation with $lookup, $unwind, and $facet
- Compound indexes, explain("executionStats"), and TTL indexes
- Backup and restore
- Basic production concepts through MCQs

Important: Complete the tasks in order because later tasks use data changed by earlier tasks.

Environment note: You do not need to configure a replica set or sharded cluster. Transactions, replication, sharding, change streams, and security are assessed through simple MCQs.

Task 1: Schema Validation and Data Creation

Step 1: Select a Fresh Database

Use a fresh database named `company_advanced`. Do not reuse data from earlier MongoDB exercises.

Step 2: Create the Validated Collection

Create an employees collection with $jsonSchema validation. The collection must require:

- `name` as a string
- `departmentId` as an integer
- `experience` as an integer
- `active` as a Boolean

Step 3: Insert the Starting Dataset

Insert the following three employee documents:

```
[
  {
    _id: 1,
    name: "John",
    departmentId: 10,
    skills: ["Java", "MongoDB"],
    experience: 4,
    active: true,
    certifications: [
      { name: "MongoDB", status: "Expired", expiryYear: 2026 }
    ]
  },
  {
    _id: 2,
    name: "Alice",
    departmentId: 10,
    skills: ["Python", "MongoDB"],
    experience: 6,
```

```
    active: true,
    certifications: [
      { name: "MongoDB", status: "Active", expiryYear: 2028 }
    ]
  },
  {
    _id: 3,
    name: "David",
    departmentId: 20,
    skills: ["Communication"],
    experience: 3,
    active: false,
    certifications: [
      { name: "Communication", status: "Active", expiryYear: 2027 }
    ]
  }
]
```

Step 4: Test the Validation Rule

Attempt to insert the invalid document below. It must be rejected because the field types do not match the validator.

```
{
  _id: 99,
  name: 123,
  departmentId: "HR",
  experience: "one",
  active: "yes"
}
```

Verify that:

```
Valid employee documents: 3
Invalid document: rejected by schema validation
```

Task 2: Advanced Array Queries and Updates

Use the same employees collection.

2.1 Find One Matching Array Object

Use `$elemMatch` to find employees whose certifications contain the same object with `name: "MongoDB"` and `status: "Active"`.

2.2 Update One Matching Array Element

Use the positional `$` operator to change John's MongoDB certification status from `Expired` to `Active`.

2.3 Update Selected Array Elements

Use `arrayFilters` to change the status to `Renewal Due` for certification elements whose `expiryYear` is less than `2027`.

Final required certification values:

```
Alice → MongoDB, Active
John  → MongoDB, Renewal Due
```

Task 3: Bulk Write Operations

Use one `bulkWrite()` call to complete all three operations:

1. Increase John's experience by `1`.
2. Set Alice's `active` value to `false`.
3. Insert Emma using the document below.

```
{
  _id: 4,
  name: "Emma",
  departmentId: 20,
  skills: ["Excel"],
  experience: 2,
  active: true,
  certifications: [
    { name: "Excel", status: "Active", expiryYear: 2026 }
  ]
}
```

Verify the final state:

```
John.experience = 5
Alice.active = false
Employee document count = 4
```

Task 4: Advanced Aggregation

Step 1: Create the Departments Collection

Insert the following documents into a departments collection:

```
[
  { _id: 10, name: "Engineering" },
  { _id: 20, name: "HR" }
]
```

4.1 Join Employees with Departments

Build an aggregation pipeline using `$lookup` and `$unwind`.

- Join employees.departmentId with departments._id.
- Return employee name and departmentName only.
- Sort by employee name in ascending order.

4.2 Count Employees for Each Skill

Use `$unwind` on `skills`, group by skill, count employees, and sort by count descending. For equal counts, sort the skill name ascending.

4.3 Return Active Employees and a Count

Use `$facet` to return both active employee names and the total active employee count in one pipeline. Sort names ascending.

Task 5: Indexing and Query Performance

5.1 Create and Verify a Compound Index

Create an ascending compound index on `departmentId` followed by `name`. Verify it using `getIndexes()`.

Required index name:

```
departmentId_1_name_1
```

5.2 Read explain("executionStats")

Run explain("executionStats") for this query:

```
db.employees
  .find({ departmentId: 10 })
  .sort({ name: 1 })
  .explain("executionStats")
```

Record the following values in answers.md:

- `winning plan stage: COLLSCAN or IXSCAN`
- `nReturned`
- `totalDocsExamined`
- `totalKeysExamined`

Note: With a very small dataset, MongoDB may choose COLLSCAN even when an index exists. Report the actual result shown on your machine.

5.3 Create a TTL Index

Create a sessions collection, insert the document below, and create a TTL index on expiresAt with expireAfterSeconds: 0.

```
{
  _id: 1,
  userName: "John",
  expiresAt: new Date(Date.now() + 600000)
}
```

Required TTL index name:

```
expiresAt_1
```

Do not wait for the document to disappear. TTL cleanup runs in the background and may not happen immediately.

Task 6: Backup and Restore

Use mongodump and mongorestore to complete the following steps:

1. Back up `company_advanced` to `company_advanced.archive` using gzip.

2. Restore it into a new database named `company_advanced_restore`.
3. Verify the restored collection counts.

Important: Restore into the new database only. Do not overwrite the working database.

Task 7: MongoDB Advanced Concepts — MCQs

Select one correct answer for each question.

Q1. Why is schema validation useful?

A. It automatically creates backups.
B. It rejects documents that do not follow required fields or data types.
C. It distributes data across shards.
D. It creates indexes for every field.

Q2. What does $elemMatch help verify?

A. Conditions match different documents.
B. Multiple conditions match the same array object.
C. Every array element is updated.
D. A collection exists.

Q3. What is the purpose of arrayFilters?

A. To sort an array.
B. To delete a collection.
C. To update selected array elements that match a condition.
D. To create a replica set.

Q4. When is bulkWrite() useful?

A. When several write operations should be sent together.
B. Only when reading data.
C. Only when creating indexes.
D. When configuring sharding.

Q5. Which stage joins documents from another collection?

A. $group
B. $facet
C. $lookup
D. $count

Q6. What does IXSCAN usually indicate in explain() output?

A. MongoDB scanned every document without an index.
B. MongoDB used an index.
C. MongoDB restored a backup.
D. MongoDB started a transaction.

Q7. What happens when abortTransaction() is called?

A. All transaction changes are committed.
B. Only the last operation is kept.
C. Uncommitted transaction changes are rolled back.
D. The database is deleted.

Q8. What does write concern w: "majority" mean?

A. Only the client confirms the write.
B. A majority of voting data-bearing replica-set members acknowledge the write.
C. Every read goes to a secondary.
D. The write is sent to every shard.

Q9. What normally happens when a replica-set Primary fails?

A. The complete database is deleted.
B. The application must manually copy all data.
C. Eligible members hold an election for a new Primary.
D. Every Secondary becomes Primary.

Q10. Which is the best direction for choosing a shard key?

A. A field with only two possible values.
B. A high-cardinality field that distributes traffic and matches common queries.
C. Any field used only for display.
D. A field that sends every new write to one shard.

Q11. What do Change Streams allow an application to do?

A. Listen for inserts, updates, replacements, and deletes.
B. Encrypt every field automatically.
C. Create a backup without mongodump.
D. Replace indexes with transactions.

Q12. Which is a good production-security practice?

A. Use an administrative user for the application.
B. Put the database password in source code.
C. Use TLS, restricted network access, and least-privilege credentials.
D. Allow connections from every network.

Deliverables

The trainee must submit:

1. A file named mongodb-advanced-assignment.js containing the MongoDB commands for Tasks 1–5.
2. A file named backup-restore-commands.txt containing the commands used for Task 6.
3. A file named answers.md containing the 12 MCQ answers and the explain("executionStats") values recorded in Task 5.2.
4. Screenshots showing schema validation, final array/bulk updates, aggregation results, index verification, explain metrics, and restored collection counts.
5. The GitHub PR link and branch name.
6. A short email status update after Task 1 and a completion update with the PR link and branch name.